

Compilador de Lox en Rust

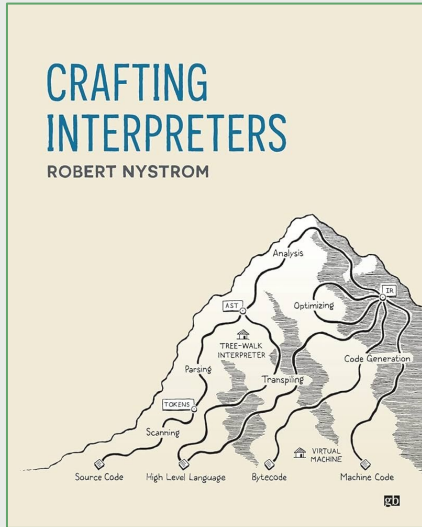
Facultad de Ingeniería,
Universidad de Buenos Aires

Mateo Julian Rico
mrco@fi.uba.ar

Alan Ezequiel Valdevenito
avaldevenito@fi.uba.ar

Bibliografía

Robert Nystrom
Crafting Interpreters



craftinginterpreters.com

A Bytecode Virtual Machine

- 14. Chunks of Bytecode
- 15. A Virtual Machine
- 16. Scanning on Demand
- 17. Compiling Expressions
- 18. Types of Values
- 19. Strings
- 20. Hash Tables
- 21. Global Variables
- 22. Local Variables
- 23. Jumping Back and Forth
- 24. Calls and Functions
- 25. Closures
- 26. Garbage Collection
- 27. Classes and Instances
- 28. Methods and Initializers
- 29. Superclasses
- 30. Optimization

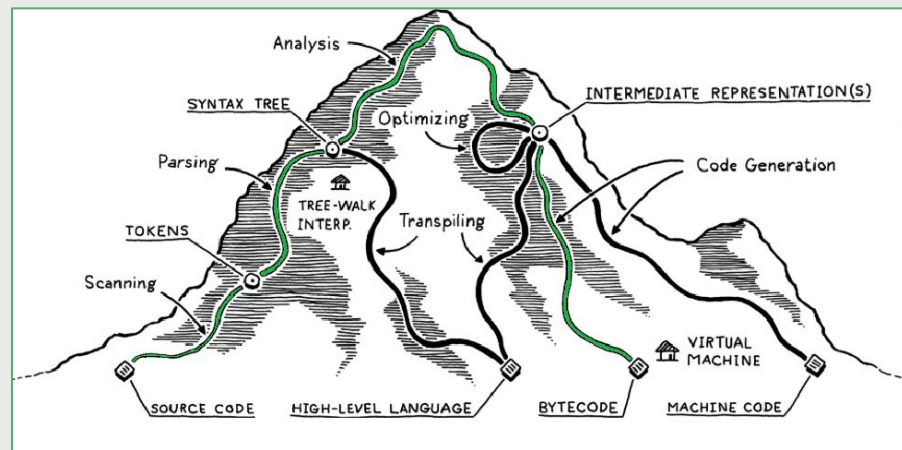


Introducción

Montañita

Nuestra implementación al tener una **fase de compilación** no ejecuta directamente, sino que **traduce** a otra representación y compila el código **generando bytecode**.

Partimos del texto escrito por el usuario, subimos por la montaña dándole estructura a lo que escribió el usuario y eventualmente llegamos a la cúpula. Desde la cúpula, comenzamos a bajar por la montaña hasta llegar a la **máquina virtual** donde se ejecuta el bytecode generado.

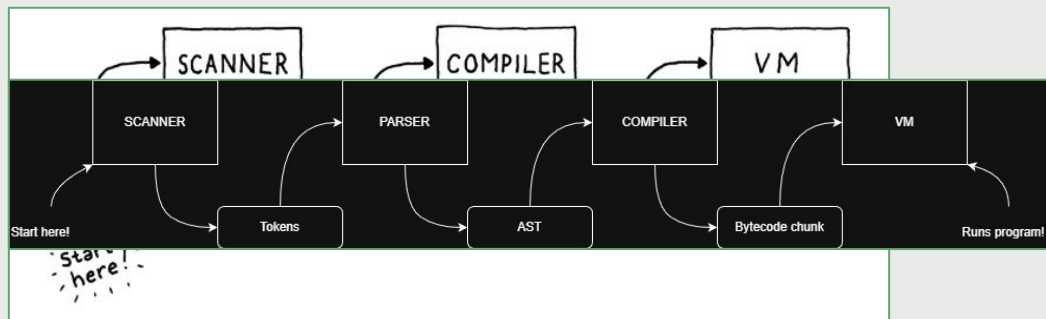


Fases

El punto de entrada en nuestra implementación es la clase **Rlox**.

Su metodo **interpret()** implementa de forma explícita las cuatro fases de nuestra implementación.

Cada una de estas fases devuelve una estructura de datos que utiliza la siguiente hasta llegar a la **VM** quien ejecuta las instrucciones dentro de un **loop**.



```
fn interpret(&mut self, source: String) {  
  
    // Phase 1: Scanning  
    let mut scanner = Scanner::new(source);  
    let tokens = scanner.scan();  
  
    // Phase 2: Parsing  
    let mut parser = Parser::new(tokens);  
    let declarations = parser.parse();  
  
    // Phase 3: Compilation  
    let mut compiler = Compiler::new();  
    let function = compiler.compile(&declarations);  
  
    // Phase 4: Running  
    let mut vm = Vm::new(function.unwrap());  
    let result = vm.run();  
}
```

Parser

Estructuras

```
pub struct Declaration {  
    pub inner: DeclarationKind,  
    pub line: usize,  
}  
  
pub enum DeclarationKind {  
    VariableDeclaration {  
        name: String,  
        initializer: Option<Expr>,  
    },  
    Statement(Statement),  
    Block(Vec<Declaration>),  
}
```

```
pub enum Statement {  
    ExprStatement(Expr),  
    PrintStatement(Expr),  
    IfStatement {  
        condition: Expr,  
        then_branch: Box<Declaration>,  
        else_branch: Option<Box<Declaration>>,  
    },  
    WhileStatement {  
        condition: Expr,  
        body: Box<Declaration>,  
    },  
    ForStatement {  
        initializer_clause: Box<Option<Declaration>>,  
        condition_clause: Option<Expr>,  
        increment_clause: Option<Expr>,  
        body: Box<Declaration>,  
    },  
    FunctionDeclaration {  
        name: String,  
        parameters: Vec<String>,  
        body: Box<Declaration>,  
    },  
    ReturnStatement {  
        result: Option<Expr>,  
    },  
}
```

```
pub enum Expr {  
    Binary {  
        left: Box<Expr>,  
        operator: Token,  
        right: Box<Expr>,  
    },  
    Unary {  
        operator: Token,  
        right: Box<Expr>,  
    },  
    Literal(Literal),  
    Grouping {  
        expression: Box<Expr>,  
    },  
    Variable {  
        name: String,  
    },  
    VariableAssignment {  
        name: String,  
        value: Box<Expr>,  
    },  
    Logical {  
        left: Box<Expr>,  
        operator: Token,  
        right: Box<Expr>,  
    },  
    Call {  
        function_identifer: Box<Expr>,  
        arguments: Vec<Expr>,  
    },  
}
```

Pratt Parser: Precedencia

Pratt Parser nos da una forma compacta de expresar precedencias sin escribir una gramática compleja.

Usa una tabla de reglas (por token) con dos partes: **prefix** (cómo empezar una expresión con ese token) e **infix** (cómo continuar si aparece ese token después de una expresión).

El **Parser** tiene un **enum Precedence** que representa la jerarquía de precedencias:

assignment < or < and < equality < comparison < term < factor < unary < call < primary

Por otro lado, la función **next()** nos da la precedencia inmediatamente mayor.

Esto corresponde a la idea del **Pratt Parser**: cada operador “sabe” cual es su precedencia y decide cuándo debe continuar o detenerse.

```
enum Precedence {  
    None,  
    Assignment, // =  
    Or,         // or  
    And,        // and  
    Equality,   // == !=  
    Comparison, // < > <= >=  
    Term,       // + -  
    Factor,     // * /  
    Unary,      // ! -  
    Call,       // . ()  
    Primary,  
}
```

```
impl Precedence {  
    fn next(&self) -> Precedence {  
        match self {  
            Precedence::None => Precedence::Assignment,  
            Precedence::Assignment => Precedence::Or,  
            Precedence::Or => Precedence::And,  
            Precedence::And => Precedence::Equality,  
            Precedence::Equality => Precedence::Comparison,  
            Precedence::Comparison => Precedence::Term,  
            Precedence::Term => Precedence::Factor,  
            Precedence::Factor => Precedence::Unary,  
            Precedence::Unary => Precedence::Call,  
            Precedence::Call => Precedence::Primary,  
            Precedence::Primary => unreachable!(),  
        }  
    }  
}
```


Pratt Parser: Reglas de parseo

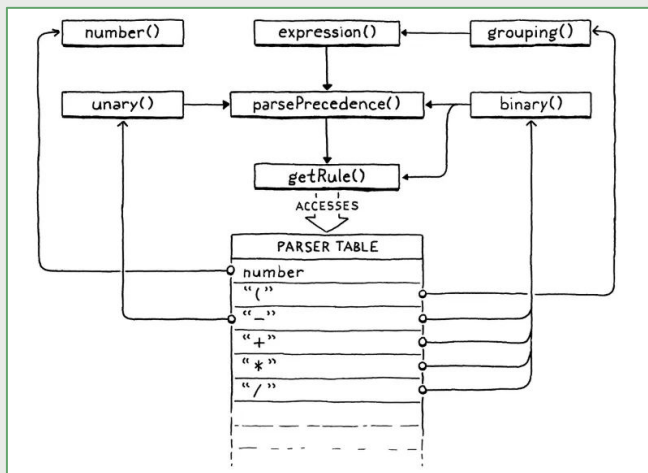
El **Parser** también tiene un **struct ParseRule** donde cada instancia es un parselet:

1. **prefix**: Cómo parsear el token si aparece al inicio de una expresión.
2. **infix**: Cómo parsear el token si aparece en medio de una expresión.
3. **precedence**: Qué precedencia tiene el operador, para decidir si continúa o se detiene.

```
struct ParseRule {  
    prefix: Option<fn(&mut Parser) -> Expr>,  
    infix: Option<fn(&mut Parser, Expr) -> Expr>,  
    precedence: Precedence,  
}
```

Pratt Parser: Tabla de reglas

El **Parser** tiene un método `get_rule(TokenType)` que representa la tabla de reglas y que devuelve el **ParseRule** (regla de parseo) para cada token. Al momento de parsear se accede al atributo **prefix** o **infix** de la estructura **ParseRule** para invocar la llamada al método de parseo correspondiente (**number**, **binary**, etc).



```
fn get_rule(token_type: TokenType) -> ParseRule {  
  match token_type {  
    TokenType::LeftParen => {  
      ParseRule::new(Some(Parser::grouping), Some(Parser::call), Precedence::Call)  
    }  
    TokenType::Plus => {  
      ParseRule::new(None, Some(Parser::binary), Precedence::Term),  
    }  
    TokenType::Star => {  
      ParseRule::new(None, Some(Parser::binary), Precedence::Factor),  
    }  
    TokenType::Number => {  
      ParseRule::new(Some(Parser::number), None, Precedence::None),  
    }  
    _ => ParseRule::new(None, None, Precedence::None),  
  }  
}
```

Pratt Parser: El corazón

Por último, el **Parser** tiene un método `parse_precedence(Precedence)` que es el corazón de **Pratt**.

Controla hasta qué nivel de precedencia se tiene que seguir consumiendo operadores.

¿Qué es lo que hace?:

1. Consume el próximo token (**advance**).
2. Identifica la regla **prefix** asociada a ese token y construye la parte inicial de la expresión (por ejemplo, un número literal, una variable o una negación).
3. Mientras el siguiente token tenga precedencia mayor o igual a la actual, consume el token y extiende la expresión aplicando su regla **infix**.

Ejemplo: Consideremos la expresión `1 + 2 * 3`.

Cuando ve `+` compara la precedencia de `*` (**Factor**) con la de `+` (**Term**).

Como **Factor** > **Term**, sigue parseando el `2 * 3` antes de cerrar el `+`.

Esto reemplaza a las funciones (`term()`, `factor()`, ...) de un **recursive-descent**. En vez de escribir manualmente la precedencia en niveles de funciones, el **Parser** usa esta comparación de precedencias.

```
fn parse_precedence(&mut self, precedence: Precedence) -> Result<Expr, ParseError> {
    self.advance();

    if self.previous.is_none() {
        self.previous = self.current.clone();
    }

    let previous_token_type = self.previous.clone().unwrap().kind;
    let Some(prefix_rule) = get_rule(previous_token_type).prefix else {
        self.error("Expect expression.");
        return Err(ParseError::ExpectedExpression);
    };

    let can_assign = precedence <= Precedence::Assignment;
    let mut expr = prefix_rule(self, can_assign)?;

    while precedence <= get_rule(self.current.clone().unwrap().kind).precedence {
        self.advance();
        let infix_rule = get_rule(self.previous.clone().unwrap().kind).infix;
        if let Some(rule) = infix_rule {
            expr = rule(self, expr, can_assign)?;
        }
    }

    if can_assign && self.matches(TokenType::Equal) {
        self.error("Invalid assignment target.");
        return Err(ParseError::ExpectedExpression);
    }

    Ok(expr)
}
```

Ejemplo

¿Como el Pratt Parser parsea $1 + 2 * 3$?

Nuestros tokens una vez que el **Scanner** los proceso son los siguientes:

[Number(1), Plus, Number(2), Star, Number(3), Eof]

El funcionamiento del **Parser** es el siguiente:

1. La clase **Rlox** llama al método `parser.parse()` que internamente (tras identificar que se trata de un **ExprStatement**) llama al método `expression()` que a su vez llama a `parse_precedence(Precedence)` con `Precedence::Assignment`.

```
fn interpret(&mut self, source: String) {  
    // Phase 1: Scanning  
    let mut scanner = Scanner::new(source);  
    let tokens = scanner.scan();  
  
    // Phase 2: Parsing  
    let mut parser = Parser::new(tokens);  
    let declarations = parser.parse();  
}
```

```
fn expression_statement(&mut self) -> Result<Declaration, ParseError> {  
    let expr = self.expression()?;  
    self.consume(TokenType::Semicolon, "Expect ';' after expression.");  
    let line = self.previous_token_line();  
    let declaration = Declaration::statement(Statement::ExprStatement(expr), line);  
    Ok(declaration)  
}
```

Ejemplo

2. Una vez dentro del método `parse_precedence()`:

- a. `advance()` actualiza `previous = Number(1)` y `current = Plus`.
- b. La `prefix_rule` para `Number(1)` es `Parser::number`.

Se invoca el método `number` accediendo al atributo `prefix` de la `ParseRule` que devuelve el método `get_rule`.

El método `number` devuelve un `Expr::Literal` y de esta forma tenemos `expr = Expr::Literal(1.0)`.

- c. El `while` compara la precedencia de `current = Plus (Term)` contra la precedencia que recibió por parámetro (`Assignment`). Es decir, realiza la comparación `Assignment <= Term` y como tenemos que `Term > Assignment` continua iterando.

```
fn parse_precedence(&mut self, precedence: Precedence) -> Result<Expr, ParseError> {
    self.advance();

    let previous_token_type = self.previous.clone().unwrap().kind;
    let prefix_rule = get_rule(previous_token_type).prefix;

    let can_assign = precedence <= Precedence::Assignment;
    let mut expr = prefix_rule(self, can_assign)?;

    while precedence <= get_rule(self.current.clone().unwrap().kind).precedence {
        self.advance();
        let infix_rule = get_rule(self.previous.clone().unwrap().kind).infix;
        expr = infix_rule(self, expr, can_assign)?;
    }

    Ok(expr)
}
```

Ejemplo

3. Dentro del `while`:

- a. `advance()` actualiza `previous = Plus` y `current = Number(2)`.
- b. La `infix_rule` para `Plus` es `Parser::binary(left = Literal(1,0))`.

Se invoca al método `binary` accediendo al atributo `infix` de la `ParseRule` que devuelve el método `get_rule`.

- c. En el método `binary`, toma `rule.precedence` (para `Plus` es `Term`) y pide `right = parse_precedence(rule.precedence.next())` que da como resultado `parse_precedence(Factor)`.

Esto es clave porque para el lado derecho de `+` pedimos `Factor` (precedencia más fuerte), así permitimos que `Factor` se anide en el lado derecho antes de cerrar el `+`.

```
let mut expr = prefix_rule(self, can_assign)?;

while precedence <= get_rule(self.current.clone().unwrap().kind).precedence {
    self.advance();
    let infix_rule = get_rule(self.previous.clone().unwrap().kind).infix;
    expr = infix_rule(self, expr, can_assign)?;
}
```

```
fn binary(&mut self, left: Expr, _can_assign: bool) -> Result<Expr, ParseError> {
    let operator = self.previous.clone().unwrap();
    let rule = get_rule(operator.kind);
    let right = self.parse_precedence(rule.precedence.next())?;

    Ok(Expr::Binary {
        left: Box::new(left),
        operator,
        right: Box::new(right),
    })
}
```

Ejemplo

4. Volvemos al método `parse_precedence(Precedence)` con `Precedence::Factor` para parsear el `right`:

- a. `advance()` actualiza `previous = Number(2)` y `current = Star`.
- b. La `prefix_rule` para `Number(2)` es `Parser::number`.

Se invoca el método `number` accediendo al atributo `prefix` de la `ParseRule` que devuelve el método `get_rule`.

El método `number` devuelve un `Expr::Literal` y de esta forma tenemos `expr = Expr::Literal(2.0)`.

- c. El `while` compara la precedencia de `current = Star (Factor)` contra la precedencia que recibió por parámetro `Factor`. Es decir, realiza la comparación `Factor <= Factor` y como tenemos que `Factor = Factor` continua iterando.

```
fn binary(&mut self, left: Expr, _can_assign: bool) -> Result<Expr, ParseError> {  
    let operator = self.previous.clone().unwrap();  
    let rule = get_rule(operator.kind);  
    let right = self.parse_precedence(rule.precedence.next())?;  
  
    Ok(Expr::Binary {  
        left: Box::new(left),  
        operator,  
        right: Box::new(right),  
    })  
}
```

```
fn parse_precedence(&mut self, precedence: Precedence) -> Result<Expr, ParseError> {  
    self.advance();  
  
    let previous_token_type = self.previous.clone().unwrap().kind;  
    let prefix_rule = get_rule(previous_token_type).prefix;  
  
    let can_assign = precedence <= Precedence::Assignment;  
    let mut expr = prefix_rule(self, can_assign)?;  
  
    while precedence <= get_rule(self.current.clone().unwrap().kind).precedence {  
        self.advance();  
        let infix_rule = get_rule(self.previous.clone().unwrap().kind).infix;  
        expr = infix_rule(self, expr, can_assign)?;  
    }  
  
    Ok(expr)  
}
```

Ejemplo

- d. Dentro del **while**: Es análogo a lo que vimos antes en el paso (3).
 - i. **advance()** actualiza **previous = Star** y **current = Number(3)**.
 - ii. La **infix_rule** para **Star** es **Parser::binary**.
 - iii. En el método **binary**, toma **rule.precedence** (para **Star** es **Factor**) y pide **right = parse_precedence(rule.precedence.next())** que da como resultado **parse_precedence(Unary)**.

```
let mut expr = prefix_rule(self, can_assign)?;

while precedence <= get_rule(self.current.clone().unwrap()).kind().precedence {
    self.advance();
    let infix_rule = get_rule(self.previous.clone().unwrap()).kind().infix;
    expr = infix_rule (self, expr, can_assign)?;
}
```

```
fn binary(&mut self, left: Expr, _can_assign: bool) -> Result<Expr, ParseError> {
    let operator = self.previous.clone().unwrap();
    let rule = get_rule(operator.kind);
    let right = self.parse_precedence(rule.precedence.next())?;

    Ok(Expr::Binary {
        left: Box::new(left),
        operator,
        right: Box::new(right),
    })
}
```


Ejemplo

5. Volvemos al método `parse_precedence(Precedence)` con `Precedence::Unary`. Se consume `Number(3)` que da como resultado `Literal(3.0)` y ve `current = Eof` con lo cual no entra el `while` y retorna `Literal(3.0)`.
6. Volvemos al método `binary`, se construye `Expr::Binary(left=2.0, op=*, right=3.0)` y lo retorna al `parse_precedence(Factor)` anterior del paso (4).
7. Por último, `parse_precedence(Factor)` devuelve `Expr::Binary(left=2.0, op=*, right=3.0)`.

```
fn parse_precedence(&mut self, precedence: Precedence) -> Result<Expr, ParseError> {
    self.advance();

    let previous_token_type = self.previous.clone().unwrap().kind;
    let prefix_rule = get_rule(previous_token_type).prefix;

    let can_assign = precedence <= Precedence::Assignment;
    let mut expr = prefix_rule(self, can_assign)?;

    while precedence <= get_rule(self.current.clone().unwrap().kind).precedence {
        self.advance();
        let infix_rule = get_rule(self.previous.clone().unwrap().kind).infix;
        expr = infix_rule(self, expr, can_assign)?;
    }

    Ok(expr)
}
```

```
fn binary(&mut self, left: Expr, _can_assign: bool) -> Result<Expr, ParseError> {
    let operator = self.previous.clone().unwrap();
    let rule = get_rule(operator.kind);
    let right = self.parse_precedence(rule.precedence.next());

    Ok(Expr::Binary {
        left: Box::new(left),
        operator,
        right: Box::new(right),
    })
}
```

Ejemplo

8. Volvemos al paso (3) donde trabajamos el **binary** del **+** y ahora tenemos **right = Expr::Binary(left=2.0, op=*, right=3.0)**.

Construye **Expr::Binary(left=1.0, op=+, right=Expr::Binary(left=2.0, op=*, right=3.0))** y lo devuelve.

9. Volvemos al paso (2) donde trabajamos el **parse_precedence(Assignment)** inicial. El método **parse_precedence(Assignment)** ve ahora **current = Eof** (cuya precedencia es **None**), compara en el **while** y termina devolviendo la expresión.

Finalmente, el **Parser** devuelve nuestro vector de **Declarations** que, para este ejemplo particular, solamente posee 1 elemento.

```
declarations = [ Declaration::statement(Statement::ExprStatement(expr), line) ]

expr = Binary(
  left = Literal(1.0),
  operator = +,
  right = Binary(left=Literal(2.0), operator=*, right=Literal(3.0))
)
```

Compiler

Estructuras

```
pub struct Chunk {  
    pub chunk: Vec<OpCode>,  
    pub constants: Vec<Value>,  
    lines: Vec<usize>,  
}
```

```
pub struct Compiler {  
    function: Function,  
    function_type: FunctionType,  
    current_line: usize,  
    locals: Vec<Local>,  
    scope_depth: usize,  
    /// Contains the identifiers of the defined global variables  
    constant_identifiers: Vec<String>,  
}
```

```
pub struct Function {  
    pub arity: usize,  
    pub chunk: Chunk,  
    pub name: String,  
}  
  
pub enum FunctionType {  
    /// Function body code  
    Function,  
    /// Top-level code  
    Script,  
}
```

Compilando

El punto de entrada de nuestro **Compiler** es el método **compile(Declarations)** que recibe como parámetro un vector de **Declarations** que obtiene del **Parser**.

El **Compiler** recorre las declaraciones y para cada una de estas identifica mediante un **match** que tipo de declaración debe compilar. Recorre el AST de forma recursiva

```
pub fn compile(declarations: &Vec<Declaration>) -> Result<Function, CompilationError> {  
    for declaration in declarations {  
        self.compile_declaration(declaration)?;  
    }  
    self.end_compiler();  
    Ok(self.function.clone())  
}
```

Al finalizar, el **Compiler** devuelve una **Function**. ¿Por que?. Porque consideramos que todo el programa está envuelto dentro de una función implícita **main()**.

Es por esto que el **Compiler** tiene una referencia a una **Function** y dentro de esta es que se encuentra el **Chunk** donde se escribe el bytecode. La idea de que el **Compiler** no apunte directamente a un **Chunk** donde escribe el bytecode surge a partir de agregar funciones a nuestra implementación.

También el **Compiler** tiene una referencia a un **FunctionType** que le permite distinguir cuando compila código a nivel superior o el cuerpo de una función.

Scopes

En el libro se manejan los scopes directamente al parsear. Mientras se parsea, al encontrar `{` se llama a **`beginScope()`** y al encontrar `}` se llama a **`endScope()`**.

Como mencionamos anteriormente, como en nuestra implementación separamos las fases de parseo y compilación, nos vimos en la necesidad de representar los scopes dentro del AST mediante los **`Declaration::Block`**.

¿Cómo funcionan los scopes en nuestra implementación con fases separadas?.

En el **Compiler** cuando nos encontramos con un **`Declaration::Block`** lo que hacemos es comenzar un nuevo scope, compilar cada declaración que contenga dentro y después finalizar el scope.

Al comenzar un nuevo scope aumentamos la profundidad. Al finalizar un scope no sólo decrementamos la profundidad, sino que también se eliminan las variables locales de este ultimo scope.

```
fn compile_declaration(declaration: &Declaration) -> Result<(), CompilationError> {
    self.current_line = declaration.line;
    match &declaration.inner {
        DeclarationKind::Block(declarations) => {
            self.begin_scope();
            for declaration in declarations {
                self.compile_declaration(declaration)?;
            }
            self.end_scope();
        }
        ...
    }
    Ok(())
}
```

```
fn begin_scope(&mut self) {
    self.scope_depth += 1;
}

fn end_scope(&mut self) {
    self.scope_depth -= 1;

    // Remove local variables from the last scope
    while let Some(local) = self.locals.last() && local.depth > self.scope_depth {
        self.locals.pop();
        self.emit_byte(Opcode::Pop, 0);
    }
}
```

Control de flujo

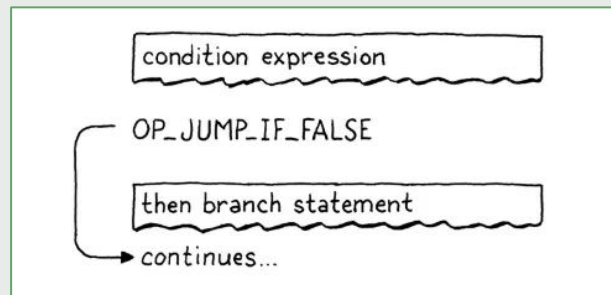
La ejecución en la **VM** se realiza normalmente incrementando el valor del **instruction pointer**, pero podemos modificar esta variable como queramos.

Para agregar control de flujo a nuestro compilador de Lox simplemente tuvimos que modificar el valor del **instruction pointer** de formas más interesantes.

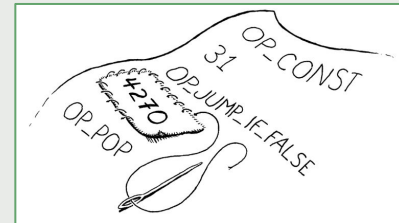
Para omitir un fragmento de código asignamos al **instruction pointer** la dirección de la instrucción de bytecode que sigue a ese código.

Para omitir condicionalmente un fragmento de código necesitamos una instrucción que consulte el valor en la parte superior del **stack**. Si es **false** se agrega un desplazamiento al **instruction pointer** para omitir un rango de instrucciones. Si es **true** no hace nada y permite que la ejecución continúe con la siguiente instrucción.

Ejemplo: Cuando nos encontramos con un **if** primero se compila la condición que se encuentra entre paréntesis. En la **VM** se deja el valor de la condición en la parte superior del **stack** y se usa para determinar si se ejecuta la rama **then** o si se omite. Después de compilar la condición, se emite una instrucción **OP_JUMP_IF_FALSE** que contiene cuanto hay que incrementar el **instruction pointer** para saltar. Si la condición es **false** se ajusta el **instruction pointer** con la cantidad que indica la instrucción.



Control de flujo



En este punto nos encontramos con un problema: Al emitir la instrucción `OP_JUMP_IF_FALSE` no sabemos cuánto hay que saltar porque todavía no se compiló la rama **then**.

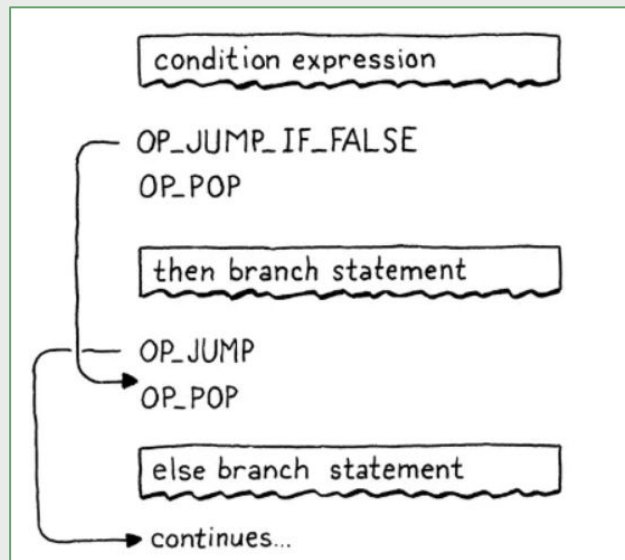
Solución: **backpatching**.

Justo después de compilar la condición, el **Compiler** genera la instrucción `OP_JUMP_IF_FALSE` pero que por el momento no contiene cuanto hay que incrementar el **instruction pointer**. El **Compiler** guarda la posición de la instrucción incompleta para volver más tarde a completar cuanto hay que incrementar el **instruction pointer**.

Continúa compilando la rama **then**. Una vez hecho esto el **Compiler** sabe cuanto debe saltar, con lo cual se vuelve a la instrucción `OP_JUMP_IF_FALSE` y se completa con cuanto hay que incrementar el **instruction pointer** para saltar hasta la rama **else**.

Si se tiene una rama **else** entonces se emite la instrucción `OP_JUMP` para evitar ejecutar ambas ramas. De igual forma, no contiene cuanto hay que incrementar el **instruction pointer**.

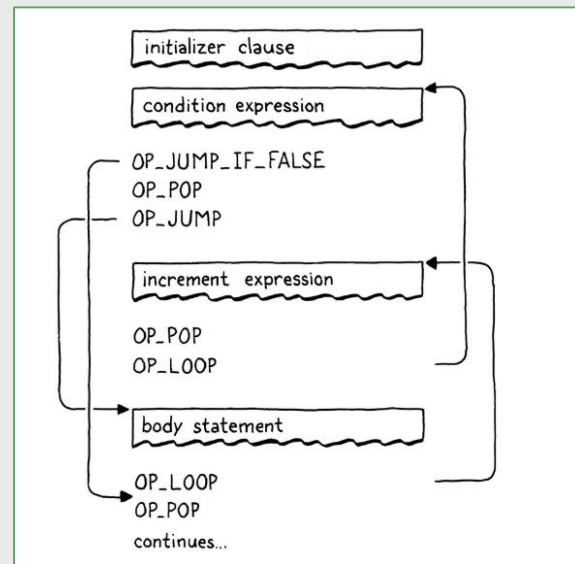
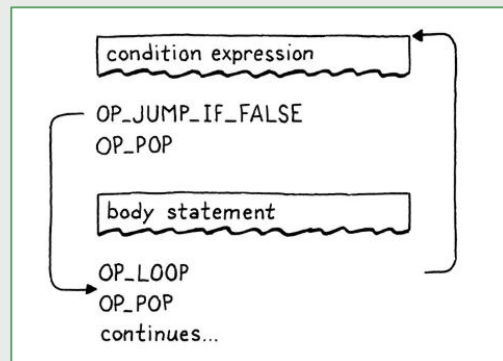
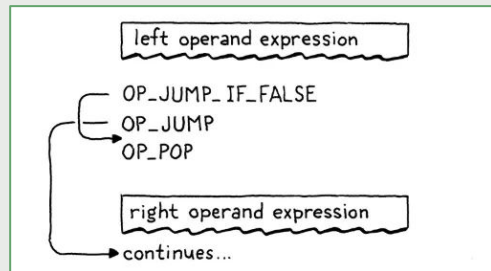
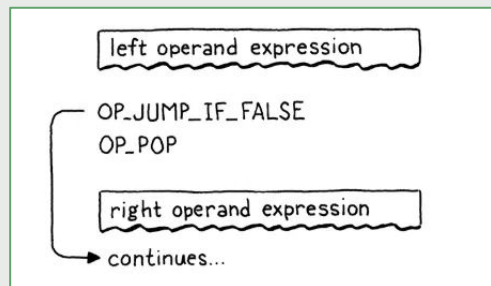
Continúa compilando la rama **else**. Se vuelve a la instrucción `OP_JUMP` y se completa con cuanto hay que incrementar el **instruction pointer** para no ejecutar la rama **else**.



compile if statement
patch jump

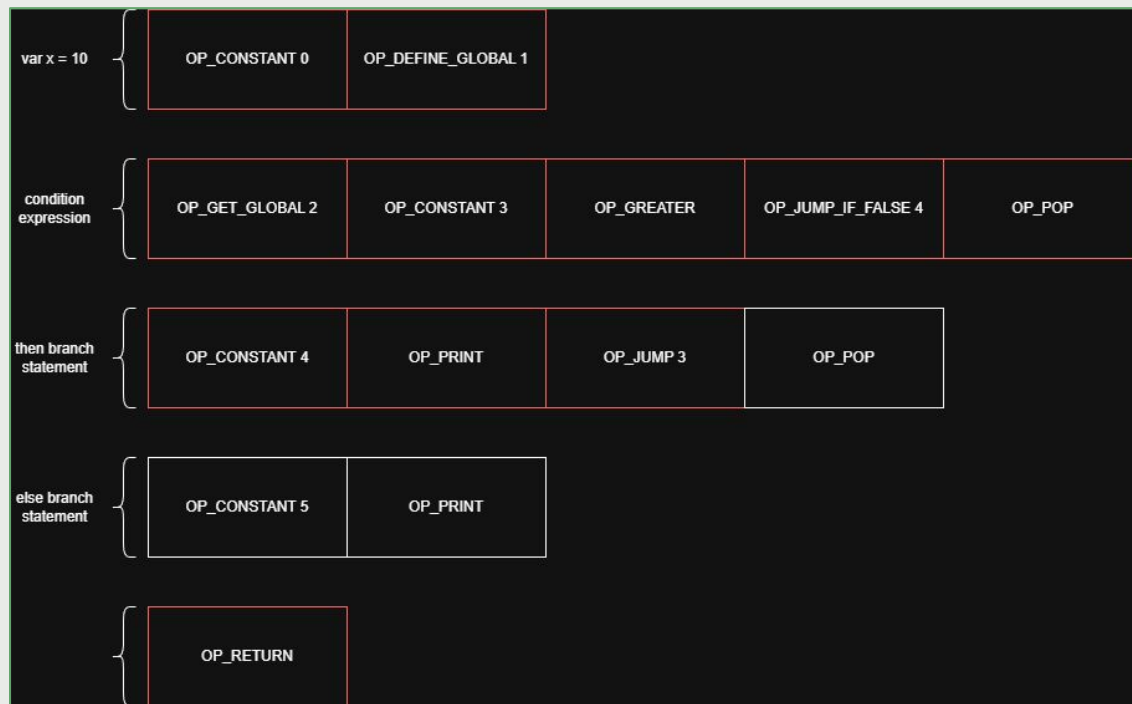
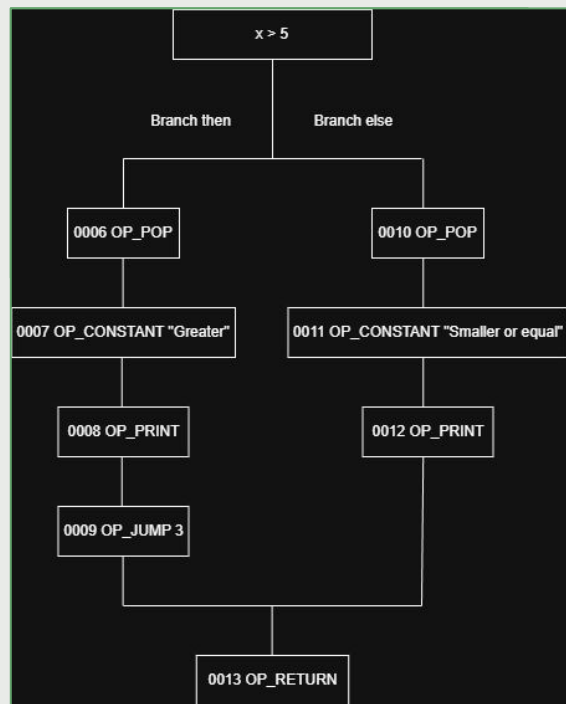
Control de flujo

Esta misma idea se utiliza para los operadores lógicos (and, or) y los ciclos (while, for).





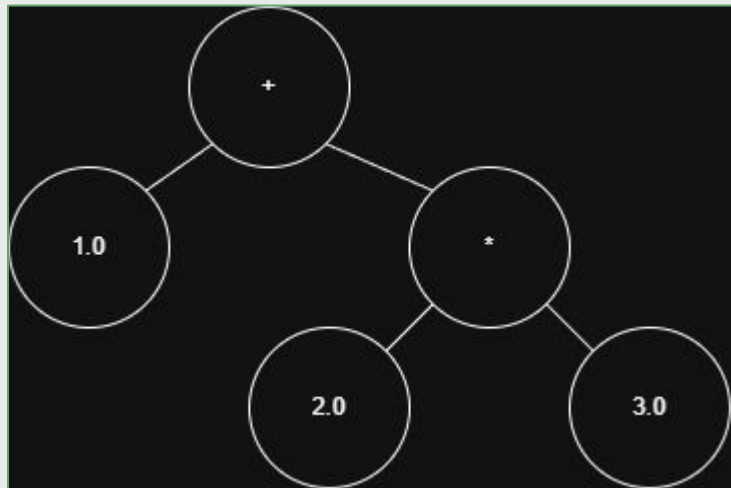
Control de flujo



Ejemplo

¿Como el **Compiler** compila $1 + 2 * 3$?

En primer lugar, recibe del **Parser** un vector con un único elemento.



```
declarations = [ Declaration::statement(Statement::ExprStatement(expr), line) ]

expr = Binary(
  left = Literal(1.0),
  operator = +,
  right = Binary(left=Literal(2.0), operator=*, right=Literal(3.0))
)
```

Ejemplo

Comienza viendo que tipo de **Declaration** es, para el ejemplo que proponemos se trata de un **DeclarationKind::Statement** y es un **ExprStatement**.

Tras esto, compila el AST de forma recursiva.

```
fn compile_declaration(&mut self, declaration: &Declaration) -> {
    self.current_line = declaration.line;
    match &declaration.inner {
        DeclarationKind::Statement(Statement::ExprStatement(expr)) => {
            self.compile_expr(&expr)?;
            self.emit_byte(OpCode::Pop, self.current_line);
        }
        ...
    }
    Ok(())
}
```

Ejemplo

```
fn compile_binary(operator: &Token, left: &Expr, right: &Expr) {
    self.compile_expr(left)?;
    self.compile_expr(right)?;

    match operator.kind {
        TokenType::Plus => self.emit_byte(OpCode::Add, operator.line),
        TokenType::Star => self.emit_byte(OpCode::Multiply, operator.line),
        ...
        _ => unreachable!(),
    }
    Ok(())
}

fn compile_literal(&mut self, literal: &Literal) {
    let line = self.current_line;
    match literal {
        Literal::Number(value) => {
            self.emit_constant(Value::Number(*value), line);
        }
        ...
    }
}
```

```
fn emit_byte(&mut self, byte: OpCode, line: usize) {
    // Escribe en el Chunk la instruccion
    self.current_chunk().write(byte, line);
}

fn emit_constant(&mut self, value: Value, line: usize) {
    // Agrega al pool de constantes del Chunk
    let constant_index = self.make_constant(value);
    // Escribe en el Chunk la instruccion
    self.emit_byte(OpCode::Constant(constant_index), line);
}
```

```
fn compile_expr(&mut self, expr: &Expr) -> {
    match expr {
        Expr::Binary { left, operator, right, } => {
            self.compile_binary(operator, left, right)?;
        }
        Expr::Literal(literal) => {
            self.compile_literal(literal);
        }
        ...
    }
    Ok(())
}
```

Ejemplo

Finalmente, el **Compiler** devuelve nuestro AST compilado dentro de un **Chunk** donde tenemos las **instrucciones** y el **pool de constantes**.



Virtual Machine

CallFrame

Dentro de la VM podemos encontrar el **stack** de valores y un vector de **CallFrames**.

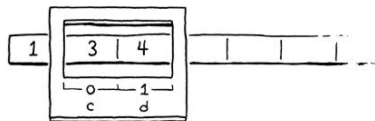
¿Qué es un **CallFrame**? Representa la invocación de una función.

Es debido a los **CallFrames** que la VM puede ejecutar múltiples funciones anidadas, recursión y mantener variables locales sin que se mezclen entre las funciones.

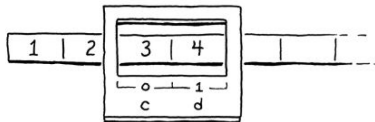
La idea que menciona el libro es que cada vez que una función se ejecuta, se obtiene una especie de **ventana** dentro del **stack** de la VM donde se pueden encontrar las variables locales.

```
fun first() {  
  var a = 1;  
  second();  
  var b = 2;  
  second();  
}  
  
fun second() {  
  var c = 3;  
  var d = 4;  
}  
  
first();
```

first call to second():



second call to second():



```
pub struct Vm {  
  stack: Vec<Value>,  
  frames: Vec<CallFrame>,  
  pub globals: HashMap<String, Value>,  
}
```

```
pub struct CallFrame {  
  pub function: Function,  
  instruction_pointer: usize,  
  slot_index: usize,  
}
```


Funciones

Declaración de funciones

En nuestra implementación, la **declaración de funciones** implica declarar una **variable global**, que **mapea** el **nombre** de la función a un **struct Function**, que contiene el **bytecode** y la **aridad** de la función.

Invocación de funciones

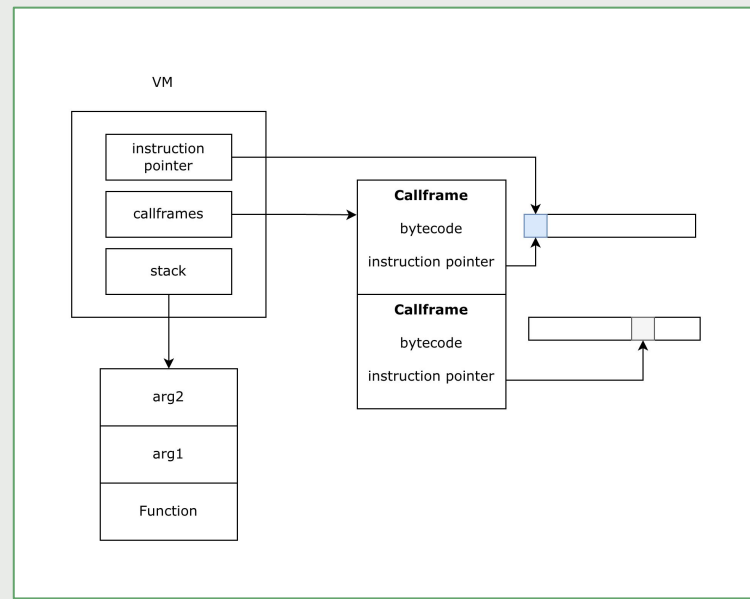
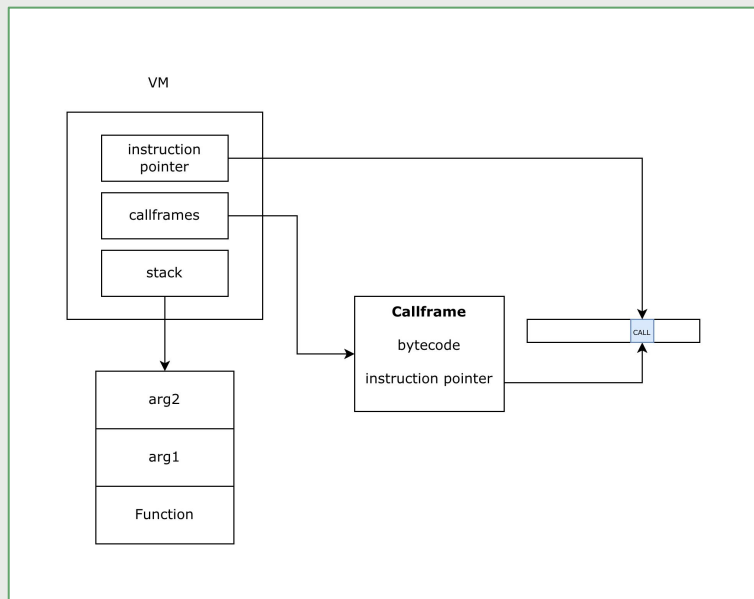
Por otro lado, al invocar una función, en la **VM** se instancia un nuevo **CallFrame**, que contiene todo el contexto de ejecución de la función:

1. El **bytecode**
2. Los **argumentos**
3. El **instruction pointer**

En cada ciclo de la **VM**, se ejecuta la instrucción a la cual apunta el **instruction pointer** del **último CallFrame**. Los argumentos de la función se pushean al **stack**. Cuando la función **retorna**, se **elimina el CallFrame**, y se **descartan las variables locales** de la función del stack.

Invocación de funciones

Los siguientes diagramas ilustran el funcionamiento interno de la **VM** al invocar una función. Se instancia un nuevo **CallFrame**, y se actualiza el **instruction pointer** para comenzar la ejecución de la función invocada.



Ejecución

Cuando la VM arranca, recibe la **Function** principal, que como mencionamos anteriormente representa el programa envuelto en una función implícita **main()**. En su constructor crea el primer **CallFrame** asociado a esta **Function**, inicializa su **instruction pointer** en 0 e inicializa su **slot index** con la posición actual del **stack**.

La ejecución sucede en el método **run()** donde tenemos un **loop** que ejecuta las instrucciones. Durante la ejecución la VM siempre ejecuta el **CallFrame** que se encuentra en la parte superior del vector de **frames**.

Una vez que estamos dentro del **loop**, primero obtenemos la siguiente instrucción a ejecutar utilizando el **instruction pointer** del **Chunk** del **Frame** actual. Tras esto, aumentamos el **instruction pointer** para que apunte a la siguiente instrucción.

Una vez que tenemos la **instruction** a ejecutar, buscamos con que instrucción hace **match** para ejecutarla.

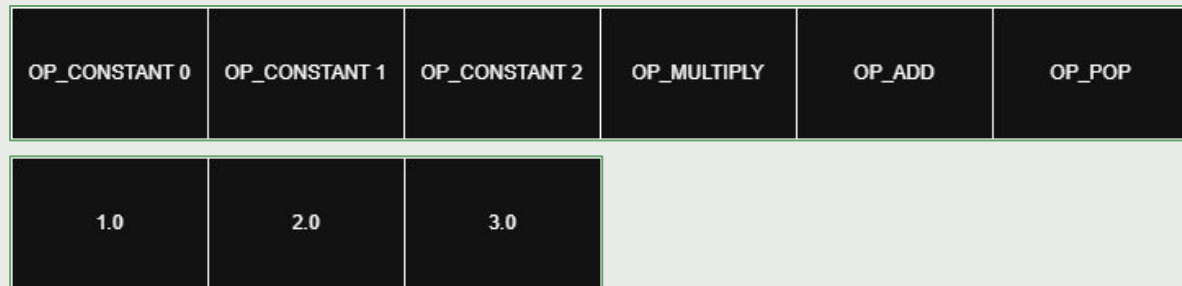
```
pub fn run(&mut self) -> Result<(), VmError> {
    loop {
        ...

        match instruction {
            OpCode::Constant(constant_index) => {
                let constant = {
                    let frame = self.frames.last().unwrap();
                    frame.function.chunk.constant_at(constant_index)
                };
                self.stack.push(constant);
            }
            OpCode::Add => {
                if self.stack_operands_are_numbers() {
                    self.binary_op_number(|a, b| a + b)?;
                } else if self.stack_operands_are_strings() {
                    self.concatenate_strings()?;
                }
            }
            OpCode::Multiply => {
                self.binary_op_number(|a, b| a * b)?;
            }
            OpCode::Pop => {
                self.stack.pop();
            }
            ...
        }
    }
}
```

Ejemplo

¿Como la VM ejecuta $1 + 2 * 3$?

En primer lugar, recibe del **Compiler** la **Function** que dentro contiene el **Chunk**.



Ejemplo

OP_CONSTANT 0	OP_CONSTANT 1	OP_CONSTANT 2	OP_MULTIPLY	OP_ADD	OP_POP
---------------	---------------	---------------	-------------	--------	--------

1.0	2.0	3.0
-----	-----	-----

Iteracion 1:

1. Tenemos **IP = 0** entonces la primera instrucción que leemos es **OpCode::Constant(0)**
2. Aumentamos a **IP = 1**.
3. Hacemos **match** con **OpCode::Constant(0)**.
4. Obtenemos la constante en el índice 0: **1,0**
5. Pusheamos la constante en el **stack**.

1.0

```
pub fn run(&mut self) -> Result<(), VmError> {
    loop {
        let instruction = {
            let frame = self.frames.last().unwrap();
            frame
                .function
                .chunk
                .instruction_at(frame.instruction_pointer)
        };

        {
            let frame = self.frames.last_mut().unwrap();
            frame.instruction_pointer += 1;
        }

        ...
    }
}
```

```
pub fn run(&mut self) -> Result<(), VmError> {
    loop {
        ...

        match instruction {
            OpCode::Constant(constant_index) => {
                let constant = {
                    let frame = self.frames.last().unwrap();
                    frame.function.chunk.constant_at(constant_index)
                };
                self.stack.push(constant);
            }
            OpCode::Add => {
                if self.stack_operands_are_numbers() {
                    self.binary_op_number(|a, b| a + b)?;
                } else if self.stack_operands_are_strings() {
                    self.concatenate_strings()?;
                }
            }
            OpCode::Multiply => {
                self.binary_op_number(|a, b| a * b)?;
            }
            OpCode::Pop => {
                self.stack.pop();
            }
            ...
        }
    }
}
```

Ejemplo

OP_CONSTANT 0	OP_CONSTANT 1	OP_CONSTANT 2	OP_MULTIPLY	OP_ADD	OP_POP
---------------	---------------	---------------	-------------	--------	--------

1.0	2.0	3.0
-----	-----	-----

Iteracion 2:

1. Tenemos **IP = 1** entonces la segunda instrucción que leemos es **OpCode::Constant(1)**
2. Aumentamos a **IP = 2**.
3. Hacemos **match** con **OpCode::Constant(1)**.
4. Obtenemos la constante en el índice 1: **2.0**
5. Pusheamos la constante en el **stack**.

2.0
1.0

```
pub fn run(&mut self) -> Result<(), VmError> {
    loop {
        let instruction = {
            let frame = self.frames.last().unwrap();
            frame
                .function
                .chunk
                .instruction_at(frame.instruction_pointer)
        };

        {
            let frame = self.frames.last_mut().unwrap();
            frame.instruction_pointer += 1;
        }

        ...
    }
}
```

```
pub fn run(&mut self) -> Result<(), VmError> {
    loop {
        ...

        match instruction {
            OpCode::Constant(constant_index) => {
                let constant = {
                    let frame = self.frames.last().unwrap();
                    frame.function.chunk.constant_at(constant_index)
                };
                self.stack.push(constant);
            }
            OpCode::Add => {
                if self.stack_operands_are_numbers() {
                    self.binary_op_number(|a, b| a + b)?;
                } else if self.stack_operands_are_strings() {
                    self.concatenate_strings()?;
                }
            }
            OpCode::Multiply => {
                self.binary_op_number(|a, b| a * b)?;
            }
            OpCode::Pop => {
                self.stack.pop();
            }
            ...
        }
    }
}
```

Ejemplo

OP_CONSTANT 0	OP_CONSTANT 1	OP_CONSTANT 2	OP_MULTIPLY	OP_ADD	OP_POP
---------------	---------------	---------------	-------------	--------	--------

1.0	2.0	3.0
-----	-----	-----

Iteracion 3:

1. Tenemos **IP = 2** entonces la tercera instrucción que leemos es **OpCode::Constant(2)**
2. Aumentamos a **IP = 3**.
3. Hacemos **match** con **OpCode::Constant(2)**.
4. Obtenemos la constante en el índice 2: **3.0**
5. Pusheamos la constante en el **stack**.

3.0
2.0
1.0

```
pub fn run(&mut self) -> Result<(), VmError> {
    loop {
        let instruction = {
            let frame = self.frames.last().unwrap();
            frame
                .function
                .chunk
                .instruction_at(frame.instruction_pointer)
        };

        {
            let frame = self.frames.last_mut().unwrap();
            frame.instruction_pointer += 1;
        }

        ...
    }
}
```

```
pub fn run(&mut self) -> Result<(), VmError> {
    loop {
        ...

        match instruction {
            OpCode::Constant(constant_index) => {
                let constant = {
                    let frame = self.frames.last().unwrap();
                    frame.function.chunk.constant_at(constant_index)
                };
                self.stack.push(constant);
            }
            OpCode::Add => {
                if self.stack_operands_are_numbers() {
                    self.binary_op_number(|a, b| a + b)?;
                } else if self.stack_operands_are_strings() {
                    self.concatenate_strings()?;
                }
            }
            OpCode::Multiply => {
                self.binary_op_number(|a, b| a * b)?;
            }
            OpCode::Pop => {
                self.stack.pop();
            }
            ...
        }
    }
}
```


Diferencias con respecto al libro

Clox vs Rlox

Clox vs Rlox

Implementado en C	Implementado en Rust
Compilador de una sola pasada	Separación de las fases en Parser y Compiler
Errores de tipo 'undefined variable' se detectan en tiempo de ejecución .	Errores de tipo 'undefined variable' se detectan en tiempo de compilación . Ejemplo